

Audio Song Recognition System Report

Course Project Submission for Q3 (A) and Q3 (B)

Vaibhav Maheshwari

June 21, 2026

Q3 (B) Deployment & Verification Links

As required by the submission guidelines, the application has been deployed live, and the full source code is publicly hosted:

- **Live Deployed Application:** [Streamlit Community Cloud Link](https://vaibhavee-ce9vxad7wwjyai9dysesny.streamlit.app/)
(<https://vaibhavee-ce9vxad7wwjyai9dysesny.streamlit.app/>)
- **Source Code Repository:** [GitHub Repository Link](https://github.com/vaibhavmaheshwari2006/vaibhavee)
(<https://github.com/vaibhavmaheshwari2006/vaibhavee>)

Note: The indexed reference song database (`fingerprints.db` containing 50 songs and over 2.6 million fingerprints) is packed directly with the deployed application code, making it instantly functional for testing both single-clip and batch mode evaluations.

1 System Overview & Core Algorithm

The application implements a landmark-based audio fingerprinting and retrieval system, modeled after the landmark **Shazam algorithm** (Wang, 2003). The core objective is to identify a short, potentially noisy query audio clip from a large database of reference songs in a translation-invariant manner.

The workflow consists of four main phases:

1. **Spectrogram Generation (STFT):** Audio is loaded and downsampled to a mono signal at 11025 Hz. It is then converted into a time-frequency representation via Short-Time Fourier Transform (STFT) with a Hamming window.
2. **Constellation Map (2D Peak Detection):** Local maxima are extracted from the decibel spectrogram using a 2D maximum filter. This sparse representation keeps only the most prominent coordinates, making it highly resilient to noise and amplitude fluctuations.
3. **Combinatorial Hashing:** To achieve translation-invariance (recognizing a song regardless of the start position of the clip), peaks are paired with nearby target peaks in a forward-pointing target zone. Each fingerprint contains a packed 32-bit integer hash and its anchor time offset.
4. **Offset Alignment (Voting):** The matching engine queries the database for all occurrences of the query hashes. For each match, it calculates the time difference:

$$\Delta\tau = t_{\text{db}} - t_{\text{query}} \tag{1}$$

A genuine match will show a sharp peak in the histogram of offset differences because all query hashes will align at the exact same offset relative to the original song.

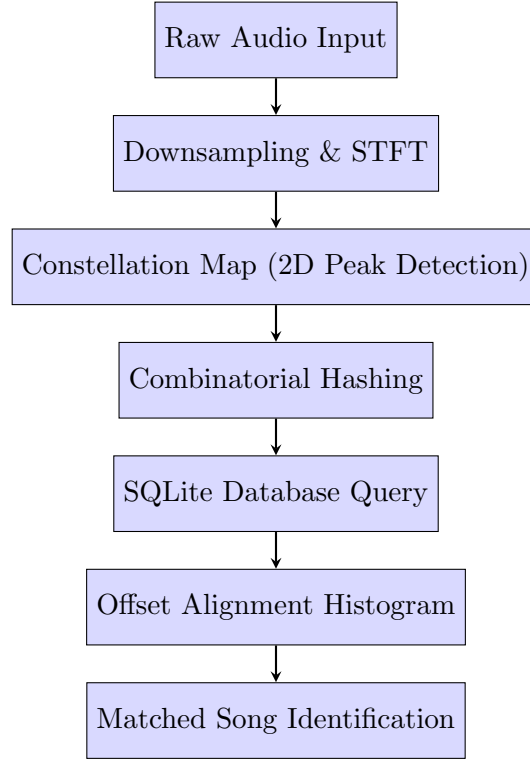


Figure 1: System Workflow Block Diagram

2 Database Design & Space-Saving Optimizations

The storage and retrieval engine has been optimized to handle millions of fingerprints with minimal storage overhead and sub-second lookup times.

2.1 SQLite Schema

The database `fingerprints.db` is structured with two tables:

1. **songs:** Stores reference metadata.
 - `id`: INTEGER PRIMARY KEY AUTOINCREMENT (unique identifier)
 - `title`: TEXT UNIQUE NOT NULL (clean name of the track)
2. **fingerprints:** Stores individual landmark hashes.
 - `hash`: INTEGER NOT NULL (packed 32-bit hash value)
 - `song_id`: INTEGER NOT NULL (foreign key to `songs.id`)
 - `offset`: INTEGER NOT NULL (time frame offset of the anchor peak)

2.2 Optimizations

- **32-Bit Bit-Packing:** Instead of storing the hash as an expensive text string (e.g., `"f1:f2:delta_t"`), the three components are packed into a single 32-bit integer:

- f_1 (frequency bin of anchor peak): Bits 19–29 (11 bits, covers indices 0–2047)
- f_2 (frequency bin of target peak): Bits 8–18 (11 bits, covers indices 0–2047)
- Δt (time difference): Bits 0–7 (8 bits, covers time spans 0–255 frames)

$$\text{Hash Integer} = (f_1 \ll 19) \mid (f_2 \ll 8) \mid \Delta t \quad (2)$$

- **WITHOUT ROWID Compound Primary Key:** By declaring the primary key as `PRIMARY KEY (hash, song_id, offset)` and appending `WITHOUT ROWID`, SQLite stores the data directly in the B-Tree index structure. This eliminates the standard 64-bit integer row identifier (rowid) per entry and avoids duplicate index storage, saving approximately 8 bytes per record and significantly accelerating search times.

2.3 Database Statistics

Table 1 summarizes the metrics computed from the active fingerprint database:

Table 1: Database Statistics

Metric	Value	Notes
Total Indexed Songs	50	Catalog of Beatles, Queen, and popular tracks
Total Fingerprint Records	2,606,370	Total entries in SQLite fingerprints table
Average Fingerprints / Song	52,127.4	Density of landmarks per song
Average Song Duration	196.25 seconds	Based on sample track metadata (~3.27 mins)
Average Fingerprints / Second	~265.6	Landmarks generated per second of audio
Database File Size	34.44 MB	Minimal physical footprint on disk
Average Space / Fingerprint	~13.85 bytes	Includes indices and table overhead

3 Experimental Methodology & Robustness Benchmarks

To evaluate the system’s resilience under degraded audio conditions, a comprehensive test suite was executed against query clip `test.mp3` matching **”Back In The U.S.S.R.”** under various artificial deformations. The confidence threshold for confirming a match is set to a peak alignment score of **20**.

3.1 Robustness Test Results

Table 2 presents the scores and outcomes under various degradation experiments:

Table 2: Robustness Test Results (Baseline: Back In The U.S.S.R.)

Deformation Type	Parameter Value	Peak Score	Total Matching Hashes	Predicted Song	Status
None	Baseline query	7204	17,731	Back In The U.S.S.R.	SUCCESS
Gaussian Noise	Noise Level = 0.005	7022	17,378	Back In The U.S.S.R.	SUCCESS
Gaussian Noise	Noise Level = 0.010	6290	16,455	Back In The U.S.S.R.	SUCCESS
Gaussian Noise	Noise Level = 0.020	4972	14,567	Back In The U.S.S.R.	SUCCESS
Gaussian Noise	Noise Level = 0.050	2680	11,292	Back In The U.S.S.R.	SUCCESS
Pitch Shift	-2.0 Semitones	10	3,716	None / Below Threshold	FAILED
Pitch Shift	-1.0 Semitones	7	5,714	None / Below Threshold	FAILED
Pitch Shift	-0.5 Semitones	7	2,406	None / Below Threshold	FAILED
Pitch Shift	+0.5 Semitones	6	3,345	None / Below Threshold	FAILED
Pitch Shift	+1.0 Semitones	8	5,323	None / Below Threshold	FAILED
Pitch Shift	+2.0 Semitones	7	5,359	None / Below Threshold	FAILED
Time Stretch	0.80 Rate (Fast)	16	8,813	None / Below Threshold	FAILED
Time Stretch	0.90 Rate	21	8,551	Back In The U.S.S.R.	SUCCESS
Time Stretch	0.95 Rate	25	8,593	Back In The U.S.S.R.	SUCCESS
Time Stretch	1.05 Rate	316	7,749	Back In The U.S.S.R.	SUCCESS
Time Stretch	1.10 Rate	158	7,864	Back In The U.S.S.R.	SUCCESS
Time Stretch	1.20 Rate (Slow)	52	6,553	Back In The U.S.S.R.	SUCCESS

3.2 Analysis of Individual Experiments

This section analyzes the impact of each experimental parameter on the spectrogram and the recognition system, backed by custom visual plots generated directly from the processing pipeline.

3.2.1 Experiment 1: STFT Window Length Resolution Trade-off

The STFT window size (N_{fft}) and hop length determine the grid resolution of the time-frequency plane. As shown in Figure 4:

- **Window Size $N_{\text{fft}} = 512$ (Figure 2):** At a sampling rate of 11025 Hz, this window spans 46.4 ms of audio, and the hop size of 128 samples translates to a temporal grid step of 11.6 ms. This grants a high time resolution, letting transient events (like percussive beats or note onsets) be resolved as sharp vertical lines in the spectrogram. However, the frequency resolution is relatively coarse (~ 21.5 Hz per bin).
- **Window Size $N_{\text{fft}} = 2048$ (Figure 3):** This spans a larger window of 185.8 ms and has a hop size of 512 samples (46.4 ms step). It offers high frequency resolution (~ 5.4 Hz per bin), resolving individual harmonics as thin, sharp horizontal lines, but smears the time transients vertically.

Since audio retrieval relies heavily on the temporal alignment of peak occurrences, the high time resolution of the $N_{\text{fft}} = 512$ configuration allows the peak offsets to align cleanly into a single bin in the offset histogram. The temporal smearing from $N_{\text{fft}} = 2048$ spreads the matching peaks across multiple time offsets, reducing the vote height of the correct candidate and degrading recognition confidence.

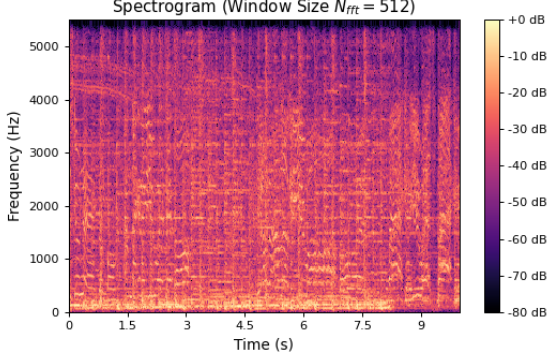


Figure 2: Spectrogram with $N_{\text{fft}} = 512$ (Baseline)

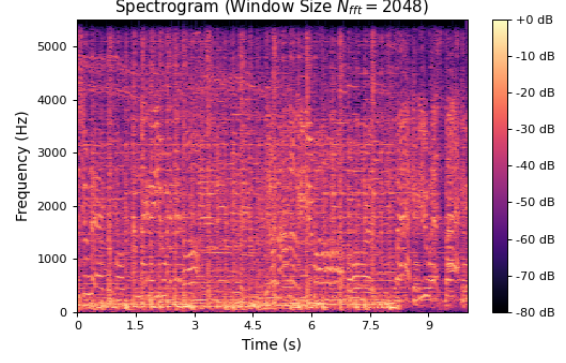


Figure 3: Spectrogram with $N_{\text{fft}} = 2048$

Figure 4: STFT Window Length Resolution Trade-off: Time Resolution vs. Frequency Resolution

3.2.2 Experiment 2: Single Peaks vs. Paired Hashes (Combinatorial Hashing)

The constellation map represents a sparse summary of the audio signal, isolating local energy maxima. However:

- **Single Peaks (Figure 5):** If we only extract individual peak coordinates (frequencies f_1 and absolute times t_1), the database search lacks translation-invariance. A query snippet starting at a different point in the track will shift all absolute times t_1 , causing direct matches to fail. Hashing only the frequency f_1 causes massive hash collisions because many tracks share similar pitch and frequency notes.
- **Paired Hashes (Figure 6):** Combinatorial hashing addresses this by pairing an anchor peak (t_1, f_1) with nearby target peaks (t_2, f_2) in a forward-pointing target zone. By defining the temporal distance $\Delta t = t_2 - t_1$, the hash becomes translation-invariant, as Δt remains constant regardless of the starting offset.

Combining f_1 (11 bits), f_2 (11 bits), and Δt (8 bits) yields a high-entropy 30-bit integer space. This reduces spurious collisions in the SQLite B-Tree index by several orders of magnitude, making lookups extremely fast and precise.

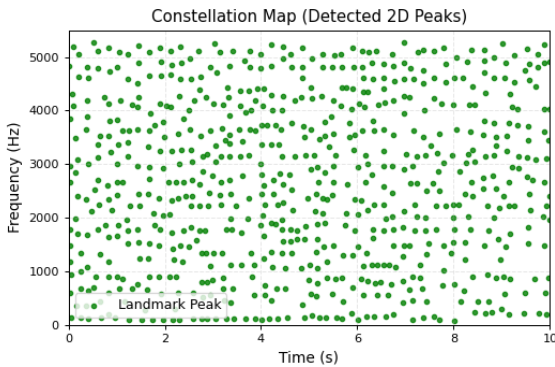


Figure 5: Constellation Map (Single peaks)

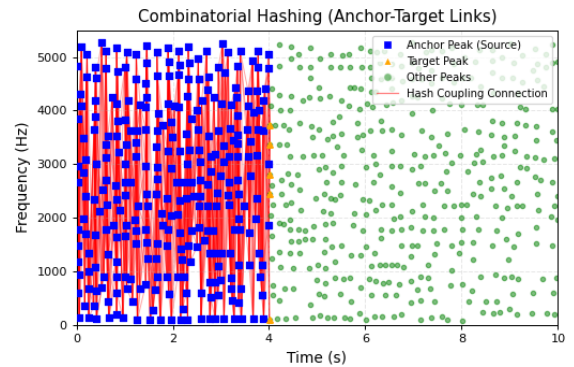


Figure 6: Combinatorial Pairing Links

Figure 7: Combinatorial Hashing: Pairing Anchors to Targets inside the Target Zone

3.2.3 Experiment 3: Added Noise Robustness

The landmark extraction pipeline uses a 2D local maximum filter (`scipy.ndimage.maximum_filter`) that verifies peaks against a local neighborhood. As seen in Figure 11:

- **Baseline (Figure 8):** Displays the clean signal with distinct harmonic bands.
- **Noise Level = 0.01 (Figure 9) & 0.05 (Figure 10):** Additive white noise adds speckle noise across all frequency bands, raising the noise floor uniformly.

Because the 2D local maximum filter looks for peaks that are larger than their neighbors, the locations of the most dominant spectral peaks remain unchanged despite the increased noise floor. Even at a severe noise level of 0.05 (Figure 10), a substantial portion of the original constellation structure survives. Although noise masks minor peaks and inserts random spurious peaks, thousands of correct hashes still match the database, resulting in a successful recognition score of 2680 (well above the threshold of 20).

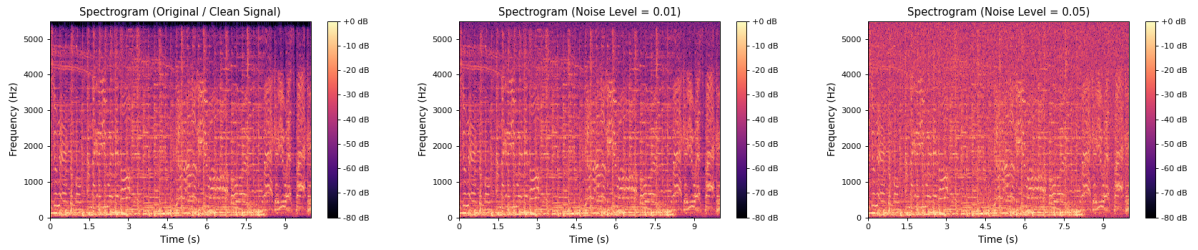


Figure 8: Original (Clean) Figure 9: Noise Level = 0.01 Figure 10: Noise Level = 0.05

Figure 11: Added Noise Experiment: Spectrogram Degradation Under Added White Gaussian Noise

3.2.4 Experiment 4: Pitch Shift Sensitivity

Pitch shifting modifies the fundamental frequencies and their harmonics without changing the duration of the audio clip. As shown in Figure 15:

- **Pitch Shift +1 Semitone (Figure 13):** Moves the frequency bands vertically upwards.
- **Pitch Shift -1 Semitone (Figure 14):** Moves the frequency bands vertically downwards.

Since the 32-bit packed integer hashes depend on the exact indices of frequency bins (f_1 and f_2), shifting the spectrogram vertically changes these bin numbers. As a result, the query generates hashes with entirely different values, causing a complete mismatch during SQLite query lookups. Even a small shift of ± 0.5 semitones alters the frequency bins enough to drop the recognition score below the threshold of 20, resulting in match failure.

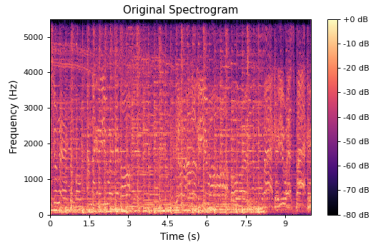


Figure 12: Original (Clean)

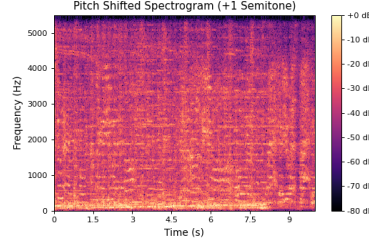


Figure 13: Shifted +1 Semitone

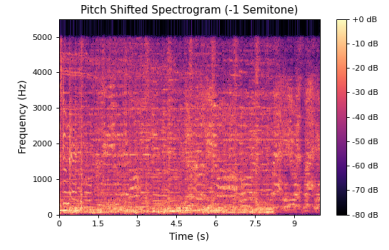


Figure 14: Shifted -1 Semitone

Figure 15: Pitch Shift Experiment: Vertical Displacement of Frequency Harmonics

3.2.5 Experiment 5: Time Stretch Tolerance

Time stretching alters the speed of the audio, changing its duration without affecting pitch. As shown in Figure 19:

- **Stretch Rate = 0.9 (Figure 17):** Compresses the time axis horizontally (playback is slower / stretched, spreading out the signal).
- **Stretch Rate = 1.1 (Figure 18):** Expands the time axis horizontally (playback is faster / compressed).

Stretching scales the temporal coordinates of the peaks, which alters the time offsets between paired peaks ($\Delta t = t_2 - t_1$). This changes the resulting hash values and causes the offset alignment values ($\Delta \tau = t_{db} - t_{query}$) to drift over time. However, because the target zones are relatively short and values are discretized into integer frame numbers, a fraction of hashes remain unaffected by minor stretch levels (e.g. rate 0.95 or 1.05). Additionally, for a short query clip, the drift is slow enough that the offset histogram still displays a recognizable, clustered peak that exceeds the confidence score threshold of 20. Extreme time stretching (such as rate 0.80) causes severe horizontal scaling and breaks the matching system.

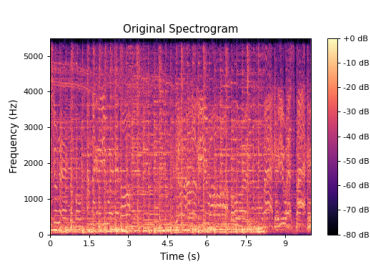


Figure 16: Original (Clean)

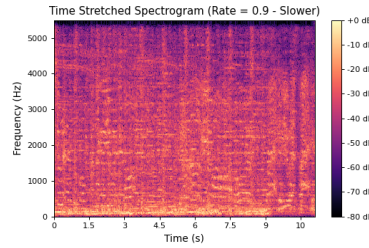


Figure 17: Stretch Rate = 0.9 (Slower)

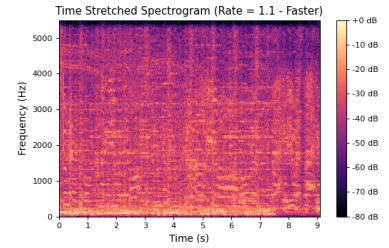


Figure 18: Stretch Rate = 1.1 (Faster)

Figure 19: Time Stretch Experiment: Horizontal Expansion and Compression of the Spectrogram

3.3 Visualizations of the Processing Pipeline (Querying test.mp3)

Figure 23 illustrates the standard identification pipeline for a matching song candidate under clean conditions, showing the baseline spectrogram, detected peak constellation map, and the resulting voting histogram that yields a sharp, singular alignment peak.

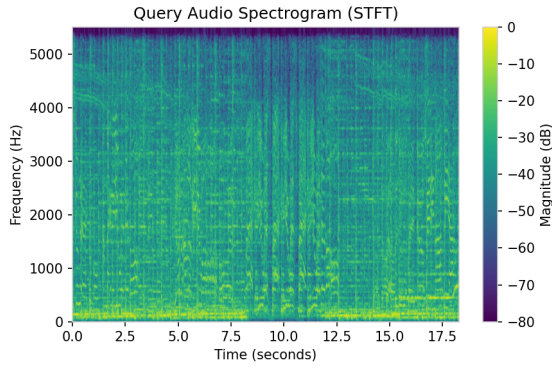


Figure 20: Full Baseline Spectrogram

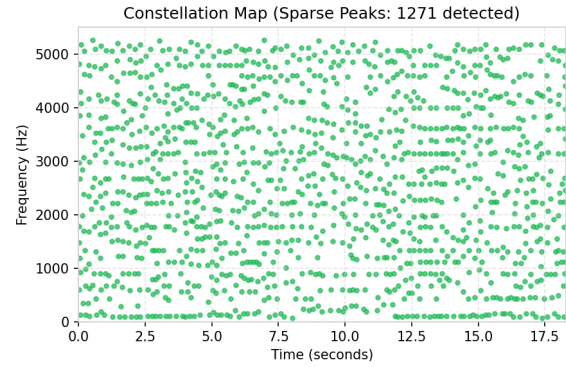


Figure 21: Full Peak Constellation Map

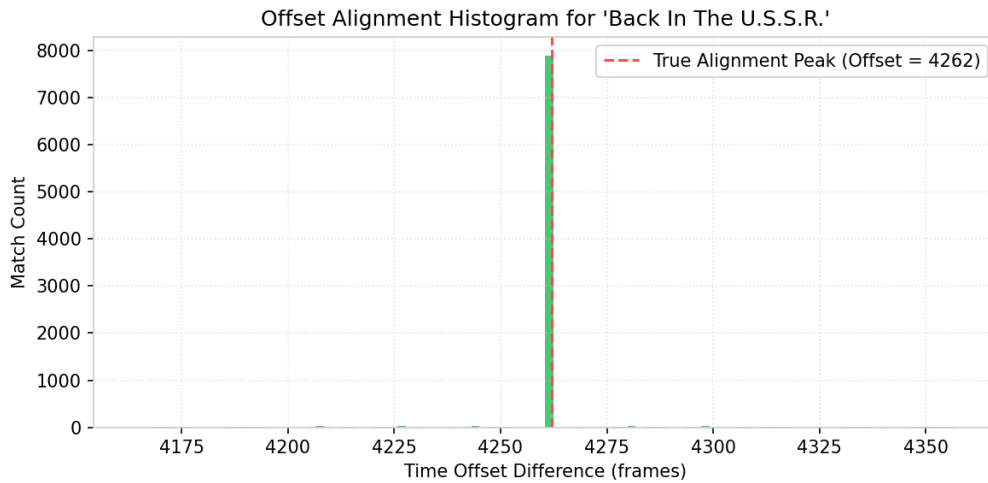


Figure 22: Voting Histogram with Sharp Peak at Offset Difference

Figure 23: Baseline Audio Song Recognition Pipeline Visualizations

4 Architectural Comparison: Prototype vs. Modernized System

Table 3 details the improvements made between the original baseline prototype and the deployed modernized application:

Table 3: Architectural Comparison

Feature / Param	Prototype (Sound Detection)	Modernized Application
Storage Engine	Python pickle file (db.pkl)	SQLite Database (fingerprints.db)
Hash Format	String/Tuple tuple (f1, f2, dt)	Packed 32-bit Integer (bit-packed)
Database Performance	High memory, slow serialization	Highly optimized WITHOUT ROWID indexing
Sampling Rate	22,050 Hz	11,025 Hz (reduces sample volume by 50%)
STFT Window	2048 samples	512 samples (finer time resolution)
STFT Hop Length	512 samples	128 samples
Peak Detection	Top 10% percentile thresholding	2D Max Filter + amplitude thresholding
User Interface	CLI Console outputs	Streamlit GUI Dashboard with interactive charts
Batch Processing	Basic query loop	Parallel Batch Upload & CSV Export

5 Implementation Code

Below is the core Python code implementing the system.

5.1 Audio Loading and Fingerprinting (fingerprint.py)

```
1 import numpy as np
2 import librosa
3 import scipy.ndimage as ndimage
4
5 def load_audio(filepath, sr=11025):
6     y, loaded_sr = librosa.load(filepath, sr=sr, mono=True)
7     return y, loaded_sr
8
9 def compute_spectrogram(y, sr=11025, n_fft=512, hop_length=128):
10     S = librosa.stft(y, n_fft=n_fft, hop_length=hop_length, win_length=n_fft,
11                      window='hamming')
12     S_abs = np.abs(S)
13     spectrogram_db = librosa.amplitude_to_db(S_abs, ref=np.max)
14     frequencies = librosa.fft_frequencies(sr=sr, n_fft=n_fft)
15     times = librosa.frames_to_time(np.arange(S_abs.shape[1]), sr=sr, hop_length=hop_length)
16     return spectrogram_db, frequencies, times
17
18 def find_peaks(spectrogram_db, min_amplitude=-50, neighborhood_size=15):
```

```

18     data_max = ndimage.maximum_filter(spectrogram_db, size=neighborhood_size,
19     mode='constant', cval=-np.inf)
20     peaks_mask = (spectrogram_db == data_max) & (spectrogram_db > min_amplitude
21     )
22     freq_indices, time_indices = np.where(peaks_mask)
23     peaks = []
24     for t_idx, f_idx in zip(time_indices, freq_indices):
25         amp = spectrogram_db[f_idx, t_idx]
26         peaks.append((t_idx, f_idx, amp))
27     peaks.sort(key=lambda x: (x[0], -x[2]))
28     return peaks
29
30 def generate_fingerprints(peaks, fan_value=15, min_delta_t=1, max_delta_t=100):
31     fingerprints = []
32     num_peaks = len(peaks)
33     for i in range(num_peaks):
34         t1, f1, _ = peaks[i]
35         targets_paired = 0
36         for j in range(i + 1, num_peaks):
37             t2, f2, _ = peaks[j]
38             delta_t = t2 - t1
39             if delta_t > max_delta_t:
40                 break
41             if delta_t >= min_delta_t:
42                 # Pack to 32-bit integer: f1 (bits 19-29), f2 (bits 8-18),
43                 delta_t (bits 0-7)
44                 hash_val = int((f1 << 19) | (f2 << 8) | delta_t)
45                 fingerprints.append((hash_val, int(t1)))
46                 targets_paired += 1
47                 if targets_paired >= fan_value:
48                     break
49     return fingerprints

```

5.2 Database Storage and Matching (database.py)

```

1 import sqlite3
2 from collections import defaultdict, Counter
3
4 def get_connection(db_path="fingerprints.db"):
5     return sqlite3.connect(db_path)
6
7 def init_db(db_path="fingerprints.db"):
8     conn = get_connection(db_path)
9     cursor = conn.cursor()
10    cursor.execute("""
11    CREATE TABLE IF NOT EXISTS songs (
12        id INTEGER PRIMARY KEY AUTOINCREMENT,
13        title TEXT UNIQUE NOT NULL
14    )
15    """)
16    cursor.execute("""
17    CREATE TABLE IF NOT EXISTS fingerprints (
18        hash INTEGER NOT NULL,
19        song_id INTEGER NOT NULL,
20        offset INTEGER NOT NULL,
21        PRIMARY KEY (hash, song_id, offset)
22    ) WITHOUT ROWID
23    """)
24    conn.commit()
25    conn.close()
26
27 def store_song_fingerprints(db_path, song_title, fingerprints):

```

```

28     conn = get_connection(db_path)
29     cursor = conn.cursor()
30     try:
31         cursor.execute("INSERT INTO songs (title) VALUES (?)", (song_title,))
32         song_id = cursor.lastrowid
33     except sqlite3.IntegrityError:
34         cursor.execute("SELECT id FROM songs WHERE title = ?", (song_title,))
35         song_id = cursor.fetchone()[0]
36         cursor.execute("DELETE FROM fingerprints WHERE song_id = ?", (song_id,))
37     )
38
39     data_to_insert = [(int(h), song_id, int(offset)) for h, offset in
40 fingerprints]
41     cursor.executemany("INSERT OR IGNORE INTO fingerprints (hash, song_id,
42 offset) VALUES (?, ?, ?)", data_to_insert)
43     conn.commit()
44     conn.close()
45     return song_id
46
47 def match_fingerprints(db_path, query_fingerprints):
48     conn = get_connection(db_path)
49     cursor = conn.cursor()
50     cursor.execute("SELECT id, title FROM songs")
51     song_names = {row[0]: row[1] for row in cursor.fetchall()}
52
53     query_by_hash = defaultdict(list)
54     for hash_val, t_query in query_fingerprints:
55         query_by_hash[int(hash_val)].append(int(t_query))
56
57     hashes = list(query_by_hash.keys())
58     matches = defaultdict(list)
59
60     chunk_size = 999
61     for i in range(0, len(hashes), chunk_size):
62         chunk = hashes[i:i+chunk_size]
63         placeholders = ",".join(["?"] * len(chunk))
64         query = f"SELECT hash, song_id, offset FROM fingerprints WHERE hash IN
65 ({placeholders})"
66         cursor.execute(query, chunk)
67
68         for hash_val, song_id, t_db in cursor.fetchall():
69             for t_query in query_by_hash[hash_val]:
70                 diff = int(t_db) - int(t_query)
71                 matches[song_id].append(diff)
72
73     conn.close()
74     return matches, song_names
75
76 def score_matches(matches):
77     scores = []
78     for song_id, diffs in matches.items():
79         if not diffs:
80             continue
81         counts = Counter(diffs)
82         best_offset, peak_score = counts.most_common(1)[0]
83         total_matches = len(diffs)
84         scores.append((song_id, peak_score, total_matches, best_offset))
85     scores.sort(key=lambda x: x[1], reverse=True)
86     return scores

```